

# inkmd

**Markdown to PDF, pure Python, zero dependencies. MIT-licensed. Deterministic.**

```
pip install inkmd
inkmd in.md -o out.pdf
```

That's the whole install. No system packages, no system fonts, no Chrome binary, no apt-get. Works the same on macOS, Linux, Windows, Alpine, AWS Lambda, a locked-down CI runner, or a Steam Deck. Color emoji are bundled, so they render anywhere without a system emoji font.

# Quarterly report — Q1 2026

A short paragraph with **bold**, *italic*, ~~struck~~, and `inline code`. This is the sort of mixed-formatting flow that inkmd handles without choking on edge cases or losing kerning between adjacent styled runs.

Revenue is up **18% YoY**. The hard part of the next quarter is shipping the renderer rewrite without slipping the integration date.

## Numbers

| Region | Q1 2025 | Q1 2026 | Change |
|--------|---------|---------|--------|
| EMEA   | £142k   | £171k   | +20.4% |
| AMER   | £208k   | £244k   | +17.3% |
| APAC   | £83k    | £98k    | +18.1% |

## Notes

- Renderer test coverage now sits at 94% line, 88% branch.
- The migration from the old XML pipeline is paused until Q2 — see the runbook at <https://example.com/runbook> for the rollback plan.

```
def quarterly_growth(prev: float, curr: float) -> float:
    return (curr - prev) / prev
```

Footnote: figures audited 2026-04-12. Contact [reports@example.com](mailto:reports@example.com) for the raw spreadsheet.

one page.

See also [examples/inkmd-brief.md](https://github.com/inkmd/inkmd/blob/master/examples/inkmd-brief.md), a two-page project brief written in inkmd-renderable markdown.

## What you get

- **A single pure-Python wheel.** No native extensions, no system libraries. Installs in under a second.
- **Faithful CommonMark plus the parts of GFM people actually use:** tables, autolinks, strikethrough, fenced code with language tags. The [supported features](#) section has the full matrix.
- **Color emoji that render anywhere.** 🚀 ✅ 🇯🇵 👍 single emoji, flags, skin tones, and ZWJ sequences (families, the rainbow flag) all render as color glyphs, inline and in table cells, from a bundled font. No system emoji font required.
- **PDFs that look right.** Real AFM-driven kerning emitted via TJ arrays, clickable links, tinted code-block backgrounds, blockquote rules that stack for nested quotes, table alignment, headings that breathe.
- **Layout you control.** Margins, body size (the whole type scale follows), line spacing, six named page sizes or custom dimensions, landscape, and forced page breaks. Wide tables fit the page losslessly: cells wrap, and a table too wide even for that splits into column panels instead of clipping.
- **Byte-identical output for the same input.** No clocks, no random IDs. Useful for version control, signed PDFs, audit trails, reproducible CI.
- **Two layers of API:** a CLI and a `compile()` / `render_file()` library function. The whole public surface is two functions and one optional config object.

## Benchmarks

Measured against WeasyPrint + markdown (the closest pure-Python alternative) on the same input documents, on the same machine. Methodology and full caveats in [BENCHMARKS.md](#); the script is at `scripts/bench.py` and is reproducible.

| Metric                       | inkmd   | Weasy Print | Ratio                          |
|------------------------------|---------|-------------|--------------------------------|
| Cold-start render, ~1 page   | 138 ms  | 731 ms      | 5.3x faster                    |
| Cold-start render, ~11 pages | 201 ms  | 1.40 s      | 7.0x faster                    |
| Peak RSS, ~11 pages          | 20 MB   | 122 MB      | 6.2x lower                     |
| Install size (venv)          | 22.3 MB | 74.6 MB     | 3.3x smaller, zero system deps |

WeasyPrint produces slightly smaller PDFs for documents over a few pages (it

compresses content streams; inkmd does not). WeasyPrint also supports full Unicode and CSS theming, which inkmd does not. inkmd does support images (PNG and JPEG embedding). The right tool depends on your input and your environment; the [comparisons doc](#) has the full picture.

## Why this exists

Markdown to PDF is a solved problem in theory and a minefield in practice. Every other tool brings heavy system dependencies that don't survive the trip into an Alpine container, a Lambda function, or a Windows machine without admin rights.

| Tool                               | What goes wrong                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------|
| <b>wkhtmltopdf</b>                 | Deprecated since 2023. Unpatched CVEs.                                                       |
| <b>Chrome headless / Puppeteer</b> | 200MB+ install. 5 to 15s cold-start latency.                                                 |
| <b>WeasyPrint</b>                  | Needs Pango, cairo, GObject (350 to 550MB of system packages). Breaks on Alpine and Windows. |
| <b>Pandoc + LaTeX</b>              | 3GB texlive install.                                                                         |
| <b>PyMuPDF-based tools</b>         | Don't build on Alpine musl.                                                                  |
| <b>borb</b>                        | AGPL, so unusable in closed-source or commercial projects without a paid licence.            |

inkmd runs anywhere Python runs: a markdown-to-PDF compiler written from scratch in pure Python, with no browser, no system libraries, and no native extensions behind it.

For the longer, honest version of how inkmd compares against every realistic alternative (including where inkmd is worse), see [docs/comparisons.md](#).

## Use cases

- **CI documentation pipelines.** Compile READMEs, release notes, or changelogs to PDF as a build artefact, in a stripped-down container, without `apt-get`.
- **Agent-generated documents.** LLM agents that need to deliver a PDF (CVs, reports, summaries) can call `inkmd.compile()` directly. No subprocess, no shell-out, no Chrome.
- **Reproducible audit trails.** Hash the markdown, hash the PDF, and the same input gives the same output bytes. Useful for compliance, signed reports, version-controlled docs.
- **Serverless rendering.** Lambda plus zero system dependencies equals a PDF endpoint that cold-starts in well under a second.
- **Restricted environments.** Locked-down CI runners, embedded hardware,

anywhere installing a 200MB browser isn't an option.

## Status

**v0.5, MIT-licensed.** 1177 tests across 49 files. Stdlib-only, Python 3.9+.  
Byte-deterministic output.

Conformance against the public spec suites, as shipped in this release: CommonMark 0.31.2 at 652/652 (100%); GFM extensions at 28/28 (100%). The full per-section breakdown is in [docs/conformance.md](#). Threat model in [docs/security.md](#). Spec-edge render samples in [docs/gallery/](#), and a real-world rendering gallery (the Ruff README, a Rust Book chapter, a Simon Willison TIL, a non-Latin scripts showcase, and inkmd's own README) in [docs/gallery/real-world/](#).

The design principle is **utter consistency**: for any markdown construct the CommonMark spec has a clear answer about, inkmd follows that answer. The conformance percentage is a proxy for "what GitHub showed you is what you get"; it isn't the goal in itself.

## Install

From PyPI:

```
pip install inkmd
```

Or grab the single-file zipapp (no pip install required). Each tagged release attaches an `inkmd.pyz` of around 230 KB that you can drop anywhere Python 3.9+ is available:

```
curl -L -o inkmd.pyz  
https://github.com/eagrede/inkmd/releases/latest/download/inkmd.pyz  
python inkmd.pyz in.md -o out.pdf
```

Or build it yourself from a checkout:

```
python scripts/build_zipapp.py    # produces dist/inkmd.pyz
```

## Usage

### CLI

```
inkmd in.md -o out.pdf           # file in, file out  
inkmd in.md > out.pdf            # file in, stdout out  
inkmd < in.md > out.pdf          # stdin in, stdout out  
inkmd in.md -o out.pdf --page-size A4 --family times  
inkmd in.md -o out.pdf --no-autolinks --no-html
```

```

inkmd in.md -o out.pdf --allow-remote-images # fetch http(s) image URLs
inkmd in.md -o out.pdf --allow-unsafe-urls   # disable URL scheme
filter
inkmd in.md -o out.pdf --emoji-fallback drop # only matters in the
no-font build
inkmd --version

```

## Library

```

import inkmd

# Compile markdown text to PDF bytes
pdf_bytes = inkmd.compile(md_text)

# Or convert files directly
inkmd.render_file("in.md", "out.pdf")

# Options (same on both functions)
pdf_bytes = inkmd.compile(
    md_text,
    page_size="A4",          # letter (default), legal, tabloid, a3, a4,
a5, or a (w, h) tuple in points
    family="times",         # or "helvetica" (default)
    autolinks=False,        # opt out of GFM bare-URL/email detection
    safe=True,              # URL scheme allow-list (default True)
    html=True,              # inline HTML allow-list (default True)
    allow_remote_images=False, # explicit opt-in to fetch http(s) images
    emoji_fallback="name",   # for emoji the font can't render: "name"
-> [rocket], or "drop"
)

```

## Layout control

Every layout knob is a plain keyword argument, and a `LayoutConfig` groups them when several call sites share a house style. The flat keyword always wins over the config, so a shared style can be overridden per call:

```

import inkmd

# Individual knobs
pdf = inkmd.compile(md_text, margin=54, font_size=11, line_spacing=1.3,
                    page_size="a4", orientation="landscape")

# A shared house style, overridable per call
HOUSE = inkmd.LayoutConfig(page_size="a4", margin=54, font_size=11)
pdf = inkmd.compile(md_text, layout=HOUSE)
pdf = inkmd.compile(md_text, layout=HOUSE, margin=72) # flat argument
wins

```

The defaults reproduce inkmd's classic output exactly: a call that passes no layout options renders byte-identically to earlier versions.

**Forced page breaks** use the standard CSS break div, the same convention browsers, Pandoc, and print stylesheets honor (and GitHub renders as nothing):

```
<div style="page-break-after: always"></div>
```

**Wide tables** fit the page losslessly by default: columns shrink and cell text wraps, and a table too wide even for that splits into column panels, each repeating the first column as the key and continuing the full row set below. The `table_overflow` knob selects the behavior: "wrap" (default), "shrink" (the pre-0.5 squeeze, which can clip), "warn" (shrink plus a `TableOverflowWarning`), or "error" (raise, for CI gates). `table_panel_min_chars` sets the narrowest a data column may go before a new panel opens (default 8). A table that fits renders identically in all four modes.

Emoji render as color glyphs out of the box (the font is bundled). Set the `INKMD_NO_EMOJI=1` environment variable to disable emoji rendering; emoji then take the `emoji_fallback` path ("name" gives a [rocket]-style label, or "drop"). The single-file zipapp build ships without the emoji font and behaves the same way. The zipapp also omits the bundled text font, so non-Latin scripts render as [U+XXXX] markers there; use the pip install (or `font_path=`) when you need them.

The public API is intentionally narrow: two functions, one optional config object, no state to manage. The CLI is a thin argparse wrapper around `compile()`.

## Supported markdown

### CommonMark

| Feature                                | inkmd |
|----------------------------------------|-------|
| Paragraphs with line wrapping          | Yes   |
| ATX headings (# to #####)              | Yes   |
| Setext headings (=== / ---)            | Yes   |
| Ordered lists, arbitrary start         | Yes   |
| Unordered lists (- / * / +)            | Yes   |
| Nested lists, mixed marker types       | Yes   |
| Tight vs. loose list detection         | Yes   |
| Blockquotes                            | Yes   |
| Nested and multi-paragraph blockquotes | Yes   |

| Feature                                                                                                                                                       | inkmd |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Blockquotes wrapping any block type                                                                                                                           | Yes   |
| Blockquote lazy continuation                                                                                                                                  | Yes   |
| Fenced code blocks                                                                                                                                            | Yes   |
| Code block language tag (info string)                                                                                                                         | Yes   |
| Indented code blocks                                                                                                                                          | Yes   |
| Indented code blocks inside list items                                                                                                                        | Yes   |
| Tabs preserved verbatim inside code                                                                                                                           | Yes   |
| Code spans ( <code>`code`</code> , multi-backtick)                                                                                                            | Yes   |
| Emphasis ( <i>*</i> , <i>_</i> )                                                                                                                              | Yes   |
| Strong emphasis ( <b>**</b> , <b>__</b> )                                                                                                                     | Yes   |
| Triple <b>***</b> becomes nested italic-bold                                                                                                                  | Yes   |
| Rule of 3 plus intraword-underscore                                                                                                                           | Yes   |
| Backslash escapes                                                                                                                                             | Yes   |
| Thematic breaks                                                                                                                                               | Yes   |
| Inline links <code>[text](url)</code>                                                                                                                         | Yes   |
| Inline link titles <code>"..."</code> , <code>'...'</code> , <code>(...)</code>                                                                               | Yes   |
| Angle-bracket autolinks <code>&lt;url&gt;</code>                                                                                                              | Yes   |
| Reference links <code>([t][ref], [ref][], [ref])</code>                                                                                                       | Yes   |
| Reference link definitions <code>([ref]: url "title")</code>                                                                                                  | Yes   |
| Hard line breaks (two-space or backslash form)                                                                                                                | Yes   |
| Soft line breaks                                                                                                                                              | Yes   |
| HTML5 entity references ( <code>&amp;am p;</code> , <code>&amp;#42;</code> )                                                                                  | Yes   |
| Images <code>![alt](url)</code>                                                                                                                               | Yes   |
| Reference-style images <code>![alt][ref]</code>                                                                                                               | Yes   |
| Image-inside-link <code>[![badge](b.png)](/repo)</code>                                                                                                       | Yes   |
| Inline HTML allow-list ( <code>&lt;sub&gt;</code> , <code>&lt;mark&gt;</code> , <code>&lt;u&gt;</code> , <code>&lt;kbd&gt;</code> , <code>&lt;br&gt;</code> ) | Yes   |
| HTML <code>&lt;img&gt;</code> (incl. <code>width</code> + <code>align</code> , and the <code>&lt;p align&gt;&lt;img&gt;</code> figure idiom)                  | Yes   |
| Block-level raw HTML ( <code>&lt;table&gt;</code> , <code>&lt;div&gt;</code> layout, arbitrary tags)                                                          | Yes   |



## GFM extensions

| Feature                                                               | inkmd          |
|-----------------------------------------------------------------------|----------------|
| Pipe tables                                                           | Yes            |
| Table column alignments                                               | Yes            |
| Bare URL autolinks ( <code>https://...</code> , <code>www...</code> ) | Yes            |
| Bare host autolinks ( <code>host.tld/path</code> )                    | Yes            |
| Email autolinks ( <code>&lt;addr@host&gt;</code> )                    | Yes            |
| Bare email autolinks (no angle brackets)                              | Yes            |
| Bare <code>mailto:</code> / <code>xmpp:</code> schemes                | Yes            |
| Strikethrough <code>~~text~~</code> / <code>~text~</code>             | Yes            |
| Task lists - <code>[ ]</code> / <code>[x]</code>                      | Yes            |
| Disallowed-HTML filter                                                | curated subset |

## Visual output

- Clickable PDF /Link annotations on every URL, inline links and autolinks alike.
- Blue underlined link text.
- Light-grey background tint behind fenced code blocks.
- Thin grey vertical rules for blockquotes. Stacked side-by-side for nested quotes.
- Tinted table headers with full grid borders and per-column alignment.
- AFM-correct kerning emitted via TJ arrays (Helvetica and Times both fully kerned).
- Strikethrough drawn as a thin horizontal bar at glyph mid-height.

## Typography

- Helvetica family (default) or Times family. Code uses Courier.
- Letter, legal, tabloid, A3, A4, and A5 page sizes, custom dimensions, portrait or landscape, with configurable margins, body size (headings, lists, tables, and code scale with it), and line spacing.
- WinAnsi character encoding: em-dash, en-dash, curly quotes, ellipsis, most Western European glyphs.
- **Color emoji** render as inline images from a bundled font: single emoji, presentation selectors, regional-indicator flags, skin-tone modifiers, ZWJ sequences (families, the rainbow flag), and keycaps, inline and in table cells. (Bitmaps scaled to text size; they soften slightly at very large heading sizes.)
- Non-Latin scripts (Cyrillic, Greek, Latin-Extended) render through an embedded font (the bundled DejaVuSans, or any TrueType font via `font_path=`). A codepoint no available font covers (e.g. CJK, which the bundled font lacks) renders a visible `[U+XXXX]` marker rather than a silent `?`, and inkmd emits a warning. CJK full

rendering is a planned later font pack.

## Determinism

`inkmd` produces **byte-identical** PDF output for the same markdown input on every platform, every Python version, every run. No real-time clocks, no random IDs, no platform-dependent iteration order.

If you hash the markdown and the PDF, the relationship is stable forever. Useful for version-controlled documents, signed/hashed PDFs, reproducible CI builds, and audit trails.

## What `inkmd` doesn't do yet

| Feature                                                   | When  | Why                                                                                                                                                                                                                  |
|-----------------------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CJK text                                                  | later | v0.4's text-font embedding made Cyrillic, Greek, and Latin-Extended render, but the bundled font has no CJK glyphs, so CJK codepoints show a visible [U+XXXX] marker. A CJK font pack is planned for a later release |
| Right-to-left and complex scripts (Arabic, Hebrew, Indic) | v1.0  | Correct bidirectional layout and shaping. Shaping ships as an optional <code>inkmd[shaping]</code> add-on (the one part of <code>inkmd</code> that uses a dependency, because it needs one)                          |
| Headers, footers, page numbers                            | v0.6  | Needs a per-page chrome system on top of a look-ahead paginator                                                                                                                                                      |
| Table of contents, bookmarks, working internal links      | v0.6  | Generated from heading structure once the paginator can resolve page positions                                                                                                                                       |
| Syntax highlighting, footnotes, callouts, captions        | v0.8  | Document constructs the docs and report use cases expect                                                                                                                                                             |
| RGBA PNG embedding                                        | v0.9  | <code>inkmd</code> supports RGB, grayscale, and <b>indexed</b> PNG (with tRNS transparency); full RGBA alpha is queued                                                                                               |

| Feature                                        | When        | Why                                                                                    |
|------------------------------------------------|-------------|----------------------------------------------------------------------------------------|
| GIF image support                              | v0.9        | LZW decoder + palette resolution                                                       |
| Justified text, hyphenation, ligatures         | v0.9        | Typesetting polish                                                                     |
| Tagged PDF / PDF/UA accessibility              | v1.0        | Structure tree, reading order, alt text                                                |
| PDF/A archival format                          | v1.0        | OutputIntent, embedded ICC profile, deterministic document ID                          |
| Math (LaTeX-style)                             | not planned | Out of scope. Use Pandoc + LaTeX.                                                      |
| Themes / CSS                                   | not planned | Out of scope. Markdown's value is its constraints.                                     |
| Prepress (bleed/crop marks, CMYK), forms, EPUB | not planned | inkmd targets office and desktop documents, not print production or reflowable e-books |

## How it works

Four layers, each strictly above the previous:

1. `parser` is a single-pass container-aware block parser plus a CommonMark inline tokeniser. Produces a frozen-dataclass AST.
2. `render` lowers AST blocks to `RenderedBlock` records with runs, spacing, indent, decorations. Carries font and link state through inline nesting.
3. `layout` wraps runs into pages, positions each `PositionedRun` against the page coordinate system, emits background rectangles for code blocks, vertical rules for blockquotes, underline plus annotation pairs for links, and bars for strikethrough.
4. `pdf` serialises pages into PDF bytes. Text via `Tj/TJ`-with-kerning, graphics via `rg/re/f`, link annotations via per-page `/Annots` arrays.

No layer imports a higher one. The whole pipeline is around 11,200 lines of pure-Python logic (including the hand-rolled OpenType reader behind color emoji) plus 4,700 lines of

generated AFM kerning tables. That's it. For a deeper walk-through (the emphasis algorithm, AFM kerning, color-emoji bitmap extraction, determinism mechanics), see [docs/internals.md](https://inkmd.github.io/docs/internals.md). The complexity profile is in [LIZARD-AUDIT.md](https://inkmd.github.io/LIZARD-AUDIT.md).

A note on font rendering

inkmd uses PDF's **14 base fonts** (Helvetica, Times, Courier, Symbol, ZapfDingbats and their variants) for text. These are spec-mandated to be available in every conforming PDF reader, so we don't ship any font files. The output stays tiny and dependency-free.

The trade-off is that the *actual rendering* depends on which Helvetica (or Times, etc.) the reader's system provides:

- **macOS** ships Helvetica Neue (real Helvetica). Renders as designed.
- **Windows** with Adobe Reader ships real Helvetica. Renders as designed.
- **Linux** typically substitutes Nimbus Sans (URW++'s free Helvetica clone). Renders very similarly but with slightly different side bearings, so spacing between glyphs can look subtly different.
- **Mobile** (iOS / Android) ships system Helvetica or Roboto variants. Mostly fine.

The advance widths are correct everywhere (PDF readers honour the AFM-published metrics), so layout (page breaks, line wrapping, paragraph flow) is identical across systems. What varies is the precise glyph shape *within* each advance-width box, which can produce slightly different visual spacing.

For most use cases this is fine. If you need pixel-identical rendering across every system (signed or archival documents, for example), use `font_path=` to embed a TrueType font, which bundles the font outlines inside each PDF. v0.4 embeds the bundled DejaVuSans for non-Latin text automatically.

## Roadmap

The release tiers are about **what a real user sees**, not about chasing a percentage.

- **v0.1:** Proof of concept: working basic PDFs. **Shipped.**
- **v0.2:** Most sane use cases work; remaining failures are rare and defensible. CommonMark 85%, GFM extensions 71%. Adds reference links, images (PNG + JPEG + indexed PNG with transparency), **color emoji** (single, flags, skin tones, ZWJ sequences, keycaps, inline and in tables), task lists, inline HTML allow-list, hard line breaks, indented code blocks (including inside list items), URL scheme filter, tab preservation, image-inside-link.
- **v0.3:** 100% CommonMark and 100% GFM extensions. The long-tail spec-corner cases, including block-level raw HTML pass-through and HTML blocks inside list items.
- **v0.4:** Text-font embedding. Non-Latin scripts (Cyrillic, Greek, Latin-Extended) render through a bundled embedded font instead of falling back to `?`, and the embedded font makes a document look the same in every viewer. A codepoint the bundled font

does not cover (e.g. CJK) shows a visible [U+XXXX] marker; CJK full rendering is a planned later font pack. **Shipped.**

- **v0.5:** Page and layout control. Margins, font size (the whole type scale follows it), line spacing, six named page sizes plus custom dimensions, landscape orientation, forced page breaks via the standard CSS break div, and lossless fitting for very wide tables (cells wrap; a table too wide even at minimum widths splits into column panels with the key column repeated, governed by a `table_overflow` knob that also keeps the old shrink behavior available). **Shipped.**
- **v0.6:** Multi-page document structure. Running headers, footers, and page numbers; a generated table of contents; PDF bookmarks; and working internal links.
- **v0.7:** Pipeline features. Document metadata, basic frontmatter parsing, clear errors instead of tracebacks, a strict mode that reports failed images, configurable URL and HTML schemes, path and resource limits, and opt-in output compression.
- **v0.8:** Document constructs. Syntax highlighting, footnotes, GitHub-style callouts, figure and table captions, and definition lists.
- **v0.9:** Typesetting and images. Justified text, hyphenation, ligatures, full RGBA PNG and GIF, image resolution handling, watermarks, and title pages.
- **v1.0:** Right-to-left and complex scripts (Arabic, Hebrew, Indic), PDF/A archival output, and tagged-PDF accessibility. The endpoint where inkmd renders any major language and produces archival-grade output.

## Licence

MIT. See [LICENSE](#).

The bundled color-emoji font is **Noto Color Emoji** (© Google), distributed under the [SIL Open Font License 1.1](#), a separate permissive licence from inkmd's own MIT code. It is shipped unmodified. The bundled text font for non-Latin scripts is **DejaVu Sans**, under its own permissive licence ([src/inkmd/assets/fonts/DejaVuSans-LICENSE.txt](#)), also shipped unmodified.

## Acknowledgements

The 14 standard PDF fonts and their AFM metric files are public-domain artefacts published by Adobe ([adobe-type-tools/Core14 AFMs](#)). Color emoji are rendered from Google's [Noto Color Emoji](#) (SIL OFL 1.1). PDF format reference: ISO 32000-1.

## About

Built by [Dylan Moir](#). Architecture, problem decomposition, and implementation directed end-to-end through AI tooling (Claude Code), with every output reviewed. Sister projects: [Nightjar](#), an autonomous LLM agent with a defence-in-depth security architecture, and [TORCH](#), an AI-orchestrated IDE.

If `inkmd` saves you a fight with WeasyPrint or a 200MB Chrome install in your CI, a star on the repo is plenty.